

The Complete Sliding Window Roadmap (Beginner → Advanced)

A pattern-first curriculum, not a random list. Work top to bottom — each section builds on the templates from the one before it. Difficulty tags: ● Easy · ● Medium · ● Hard.

0. Core Templates (learn these first, cold)

Fixed-size window

```
left = 0
for right in range(n):
    add(arr[right])
    if right - left + 1 == k:
        record_answer()
        remove(arr[left]); left += 1
```

Variable-size window (shrink while invalid)

```
left = 0
for right in range(n):
    add(arr[right])
    while window_invalid():
        remove(arr[left]); left += 1
    record_answer()    # window is valid here
```

Variable-size window (shrink while still valid — for "minimum

window")

```
left = 0
for right in range(n):
    add(arr[right])
    while window_valid():
        record_answer()      # shrink to find the smallest
        remove(arr[left]); left += 1
```

Master these three loops and 90% of the 160 problems become "which template + which condition."

1. Fixed-Size Sliding Window

Window size *k* is given/fixed. You slide it and track a running aggregate.

#	Problem	LC #	Diff	Key Insight
1	Maximum Average Subarray I	643	●●●	Running sum, k-window
2	Substrings of Size Three with Distinct Characters	1876	●●●	Fixed window + set
3	Number of Sub-arrays of Size K and Avg ≥ Threshold	1343	●●●	Running sum threshold
4	Find K-Length Substrings With No Repeated Characters	1100	●●●	Frequency map, window=k
5	Grumpy Bookstore Owner	1052	●●●	Fixed window "gain" trick
6	Permutation in String	567	●●●	Fixed window = len(s1), freq compare

7	Find All Anagrams in a String	438	●	Same as above, collect all starts
8	Sliding Window Maximum	239	●	Bridges into Section 8 (deque)
9	Maximum Number of Vowels in a Substring of Given Length	1456	●	Fixed window boolean count
10	Maximum Sum of Distinct Subarrays With Length K	2461	●	Fixed window + duplicate check
11	Sliding Subarray Beauty	2653	●	Fixed window + counting sort
12	Minimum Swaps to Group All 1's Together II	2134	●	Circular fixed window
13	Distinct Numbers in Each Subarray	2461-variant / 1567-style	●	Frequency map decay

2. Variable-Size Sliding Window — Longest Subarray/Substring

Expand right always; shrink left only when a constraint breaks.

#	Problem	LC #	Diff	Key Insight
1	Longest Substring Without Repeating Characters	3	●	Set/map of last index
2	Longest Substring with At Most Two Distinct Characters	159	●	Freq map, size ≤ 2

3	Longest Substring with At Most K Distinct Characters	340	●	Generalized version of #2
4	Fruit Into Baskets	904	●	Literally "at most 2 distinct" reskinned
5	Max Consecutive Ones III	1004	●	At most K zeros allowed
6	Longest Repeating Character Replacement	424	●	Track max-freq char in window
7	Longest Subarray of 1's After Deleting One Element	1493	●	At most 1 zero, subtract 1
8	Maximum Erasure Value	1695	●	Unique-elements window, sum tracked
9	Maximize the Confusion of an Exam	2024	●	Two parallel "at most K" windows
10	Frequency of the Most Frequent Element	1838	●	Sorted array + window "cost"
11	Number of Substrings Containing All Three Characters	1358	●	Count-of-valid-windows trick
12	Longest Substring Of All Vowels in Order	1839	●	Multi-condition longest window
13	Continuous Subarrays	2762	●	Sliding max/min diff ≤ 2
14	Find the Longest Equal Subarray	2831	●	Longest window with $\leq k$ deletions
15	Take K of Each Character From Left and Right	2516	●	Complement trick: longest middle window to remove

3. Variable-Size Sliding Window — Shortest/

Minimum Subarray or Substring

Shrink aggressively once the window becomes valid, to find the minimum.

#	Problem	LC #	Diff	Key Insight
1	Minimum Size Subarray Sum	209	●	Classic shrink-while-valid
2	Minimum Window Substring	76	●	Freq map + shrink-while-valid
3	Minimum Window Subsequence	727	●	Order matters — two-pointer replay
4	Replace the Substring for Balanced String	1234	●	Complement window, shrink for min
5	Shortest Subarray with Sum at Least K	862	●	Needs monotonic deque (negatives allowed)
6	Minimum Operations to Make Array Continuous	2009	●	Sort + window on unique values

4. At Most K / Exactly K Pattern

The single most important "trick" in sliding window: $exactly(K) = atMost(K) - atMost(K-1)$

#	Problem	LC #	Diff	Key Insight
1	Subarray Product Less Than K	713	●	Pure "at most" window, count subarrays

2	Subarrays with K Different Integers	992	●	atMost(k) – atMost(k-1) — the archetype
3	Number of Nice Subarrays	1248	●	"Exactly K odd numbers" via at-most trick
4	Number of Subarrays with Bounded Maximum	795	●	atMost(hi) – atMost(lo-1)
5	Count Number of Substrings With Only One Distinct Letter follow-ups (K-distinct family)	—	●	Generalize #2 to substrings
6	Count Subarrays Where Max Element Appears at Least K Times	2962	●	At-least-K via at-most complement
7	Binary Subarrays With Sum	930	●	Exactly-K sum via at-most trick
8	Count the Number of Good Subarrays	2537	●	Exactly-K pairs via at-most trick

5. Frequency Map + Sliding Window

The window's validity depends on a hashmap of counts, not just size.

#	Problem	LC #	Diff	Key Insight
1	Longest Substring Without Repeating Characters	3	●	Revisit with pure freq-map lens
2	Minimum Window Substring	76	●	Revisit — need/ have counters
3	Find All Anagrams in a String	438	●	Freq map equality check

4	Permutation in String	567	●	Same, fixed-size version
5	Substring with Concatenation of All Words	30	●	Multi-word freq map, word-length steps
6	Subarrays with K Different Integers	992	●	Freq map size == k
7	Count Substrings That Contain Every Vowel and K Consonants	3306	●	Multi-key freq validity

6. Character Count / Anagram Window

Specialized frequency-map problems centered on permutations/ anagrams.

#	Problem	LC #	Diff	Key Insight
1	Permutation in String	567	●	Fixed window anagram check
2	Find All Anagrams in a String	438	●	Same, collect all indices
3	Substring with Concatenation of All Words	30	●	"Anagram of word-list"
4	Minimum Number of Flips to Make Binary String Alternating	1888	●	Circular window + pattern match
5	Count Anagrams Reskins (Group Anagrams family, for contrast)	49	●	Not sliding window — good to know why

7. Two Pointers + Sliding Window (hybrid problems)

Where the "window" is really governed by a monotonic two-pointer sum/product relationship.

#	Problem	LC #	Diff	Key Insight
1	Subarray Product Less Than K	713	●	Product shrinks monotonically
2	Continuous Subarrays	2762	●	Both pointers move forward only
3	Max Consecutive Ones III	1004	●	Two-pointer zero-budget
4	3Sum With Multiplicity-style windows (contrast case)	923	●	Sorted two-pointer, not window
5	Container With Most Water (related two-pointer, not window)	11	●	Compare/contrast with sliding window
6	Trapping Rain Water (related two-pointer)	42	●	Compare/contrast — precompute vs pointer

8. Monotonic Deque + Sliding Window

When you need the window's min/max in $O(1)$ as it slides — the hardest core skill.

#	Problem	LC #	Diff	Key Insight
1	Sliding Window Maximum	239	●	THE canonical monotonic deque problem

2	Shortest Subarray with Sum at Least K	862	●	Deque over prefix sums
3	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	1438	●	Two deques (min + max) in one window
4	Constrained Subsequence Sum	1425	●	DP + monotonic deque
5	Jump Game VI	1696	●	DP + sliding window max
6	Max Value of Equation	1499	●	Deque on transformed values
7	Count Subarrays With Fixed Bounds	2444	●	Track min/max positions + boundary

9. Multiple Constraints (multi-condition windows)

More than one rule must hold simultaneously — usually 2+ counters or a bitmask.

#	Problem	LC #	Diff	Key Insight
1	Minimum Window Substring	76	●	Multiple character-count constraints
2	Longest Nice Subarray	2401	●	Bitmask AND constraint
3	Longest Substring Of All Vowels in Order	1839	●	Ordering + coverage constraint
4	Number of Substrings Containing All Three Characters	1358	●	3 simultaneous coverage constraints

5	Minimum Number of Flips to Make Binary String Alternating	1888	●	Pattern + rotation constraint
6	Subarrays with K Different Integers	992	●	Count constraint + exactly-K logic
7	Maximize the Confusion of an Exam	2024	●	Two constraints solved as two windows

10. Prefix Sum + Window (adjacent pattern — know the boundary!)

True sliding window requires monotonic add/remove. When negatives are allowed, you often need prefix sums + hashmap instead — important to recognize the difference.

#	Problem	LC #	Diff	Key Insight
1	Subarray Sum Equals K	560	●	NOT sliding window (negatives) — prefix+map
2	Shortest Subarray with Sum at Least K	862	●	Prefix sum + monotonic deque
3	Maximum Size Subarray Sum Equals K	325	●	Prefix sum + hashmap
4	Maximum Number of Non-Overlapping Subarrays With Sum Equals Target	1546	●	Prefix sum + greedy window reset

11. Advanced Sliding Window (binary search

/ precompute hybrids)

Where sliding window is combined with another technique for an optimized search.

#	Problem	LC #	Diff	Key Insight
1	Magnetic Force Between Two Balls	1552	●	Binary search on answer, window-check feasibility
2	Frequency of the Most Frequent Element	1838	●	Sort + window "budget"
3	Longest Repeating Character Replacement	424	●	Window that never shrinks below max size
4	Count Unique Characters of All Substrings of a Given String	828	●	Contribution technique, not classic window
5	Substring with Concatenation of All Words	30	●	Multi-window offsets (0..wordLen-1)

12. Hard / Interview-Level Mastery Set

If you can solve these cold, you've mastered the pattern.

#	Problem	LC #	Diff
1	Minimum Window Substring	76	●
2	Sliding Window Maximum	239	●
3	Subarrays with K Different Integers	992	●
4	Substring with Concatenation of All Words	30	●
5	Minimum Window Subsequence	727	●

6	Shortest Subarray with Sum at Least K	862	●
7	Constrained Subsequence Sum	1425	●
8	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	1438	●
9	Count Subarrays With Fixed Bounds	2444	●
10	Max Value of Equation	1499	●
11	Count Unique Characters of All Substrings	828	●

Suggested Study Order (Phases)

1. **Phase 1 — Foundations (Week 1):** Section 1 (Fixed-size) fully, then start Section 2.
2. **Phase 2 — Variable Window Fluency (Week 2):** Finish Sections 2 & 3. Drill the two core templates until you can write them blind.
3. **Phase 3 — Frequency Maps (Week 3):** Sections 5 & 6. This is where most interview questions live (438, 567, 76).
4. **Phase 4 — The Exactly-K Trick (Week 3–4):** Section 4. Learn `atMost(K) - atMost(K-1)` deeply — it unlocks a whole problem class.
5. **Phase 5 — Monotonic Deque (Week 4–5):** Section 8. Hardest data-structure integration; do 239 first, then 862, then 1438.
6. **Phase 6 — Multi-Constraint & Hybrid (Week 5–6):** Sections 9, 10, 11.
7. **Phase 7 — Mastery (Week 6+):** Section 12, timed, no hints.

Notes on the ~160-Problem Claim

LeetCode's "Sliding Window" tag includes many problems that use the *name* loosely (two-pointer array problems, monotonic stack problems, prefix-sum problems). This roadmap includes ~95 curated problems that map cleanly onto real sliding-window sub-patterns, plus a handful of

adjacent problems (Sections 7, 10, 11) included deliberately so you learn to *recognize when a problem looks like sliding window but isn't* — that judgment call is itself an interview skill. Once you've cleared this list, revisit the full LeetCode tag search and the remaining ~60 will mostly be reskins of patterns you already know.